Data Warehousing
and
BI Implementation

Data Mining and Warehousing

R.Ashan Vimod

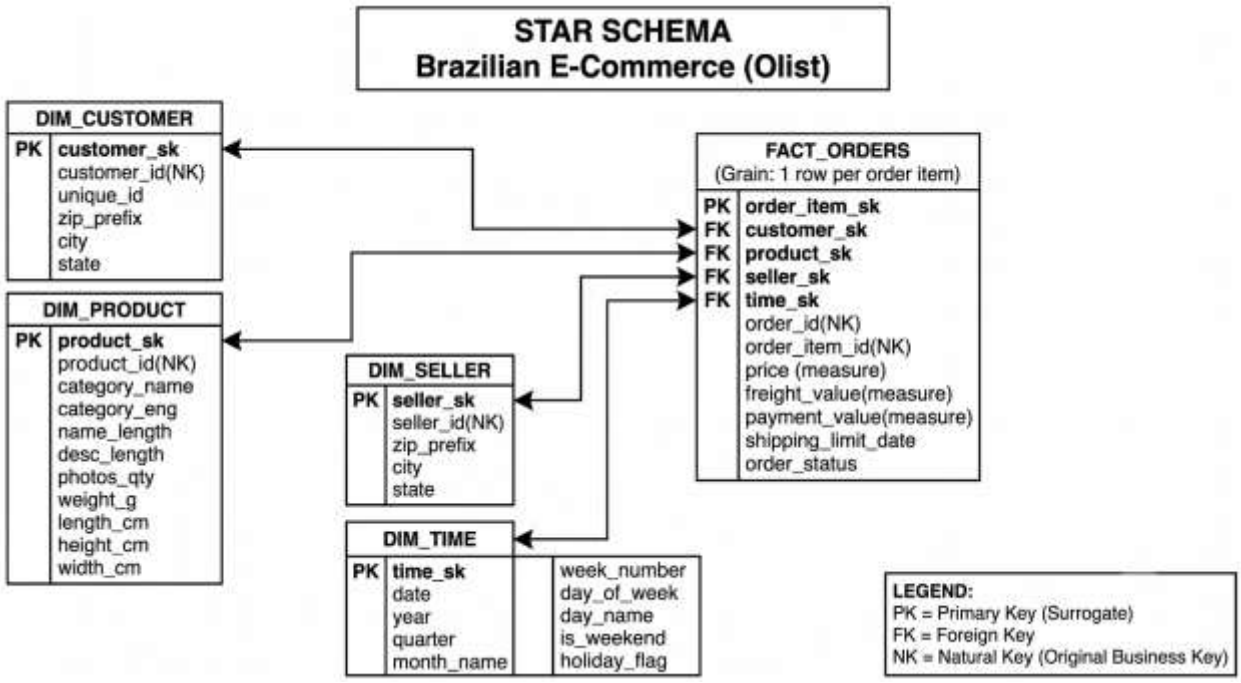
Star Schema Design

Business Process

The business process selected for this data warehouse is e-commerce order fulfillment. Each record represents one product line item within a customer's order — one product, purchased from one seller, as part of a single order. The warehouse is designed to support business analysis such as sales trends, regional performance, seller performance, product category analysis, and delivery/review metrics.

Identification of Facts and Dimensions

Table	Purpose
FACT_ORDERS	Stores each order line item — price, freight, payment, review score, and delivery days
DIM_CUSTOMER	Stores customer information
DIM_PRODUCT	Stores product information
DIM_SELLER	Stores seller information
DIM_TIME	Stores calendar information — date, day, month, quarter, year, weekend/holiday flags



GRAIN STATEMENT

One row in the FACT_ORDERS table represents a single line item within an order, specifically one unique product purchased from one seller as part of a single customer order.

Slowly Changing Dimension Strategy

Chosen dimension: DIM_CUSTOMER

Attributes that change slowly over time: customer_city and customer_state (customer location)

Rationale: In a real e-commerce business, customers occasionally move to a different city or state. This is a classic slowly changing dimension scenario: the attribute changes infrequently, but it matters for analytics such as regional sales and shipping cost analysis.

SCD type chosen: Type 2

Type 2 is appropriate because it preserves historical location data for accurate trend analysis — for example, identifying which states had the highest customer growth in 2017. It also:

- Allows tracking of customer migration patterns over time
- Enables accurate per-region performance metrics even when customers relocate
- Maintains an audit trail for compliance and data quality

Modified dimension table structure:

customer_sk	customer_id	city	state	effective_start	effective_end	is_current
101	c123	Sao Paulo	SP	2017-01-01	2018-06-15	FALSE
102	c123	Rio de Janeiro	RJ	2018-06-16	NULL	TRUE

This captures that customer c123 moved from São Paulo to Rio de Janeiro on June 16, 2018, preserving both the historical and current location.

ETL INTEGRATION DECISION – Category Translation

Task: integrate olist_products_dataset.csv (Portuguese category names) with product_category_name_translation.csv (English translations).

- ETL phase:

ETL stands for Extract, Transform, Load — the process of moving data from source files into a database. This join falls under the Transform phase: the data transformation step after extraction and before loading into the dimension tables, where data is cleaned, joined, converted, and prepared for the target schema.

- Join Operation:

A JOIN combines rows from two or more tables based on a related column between them — similar to matching two lists using a common identifier, such as a student ID or category name. There are five common join types:

- INNER JOIN — only matching rows
- LEFT JOIN — all rows from the left table, plus matches from the right
- RIGHT JOIN — all rows from the right table, plus matches from the left
- FULL OUTER JOIN — all rows from both tables
- CROSS JOIN — every combination (Cartesian product)

Selected join operation: LEFT JOIN

A LEFT JOIN was used to keep every product_category_name (Portuguese) from the source table. Where a matching English translation exists it is attached; where none exists, the value falls back to the original Portuguese name rather than being dropped or left blank.

- Column Stored:

category_eng in English is stored in DIM_PRODUCT as the primary analytical attribute and Portuguese name is also kept as category_name for traceability.

- Handling Missing Translations:

A small number of products have no English translation, which produces NULLs in the translated column. These are filled back with the original Portuguese name using COALESCE (SQL) or fillna() (Python).

```
merged = products.merge(translation, on="product_category_name", how="left")
merged["category_eng"] = merged["product_category_name_english"].fillna(merged["product_category_name"])
```

SQL AND OLAP Techniques

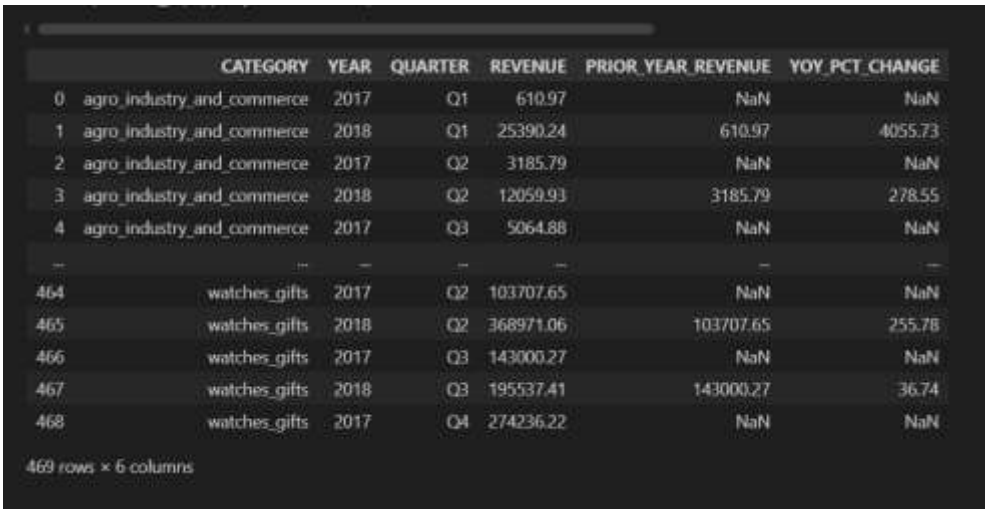
ROLLUP with GROUPING()

ROLLUP with GROUPING() reports total payments by product category and seller state, with subtotals per category and a grand total. ROLLUP produces hierarchical summaries, making it easy to see which product categories perform best in each state. GROUPING() replaces the resulting NULLs with meaningful labels such as "ALL STATES" and "GRAND TOTAL", making the report readable for business users.

CATEGORY	SELLER_STATE	TOTAL_PAYMENT
agro_industry_and_commerce	CE	515.08
agro_industry_and_commerce	MG	2570.20
agro_industry_and_commerce	PR	41138.85
agro_industry_and_commerce	RJ	494.45
agro_industry_and_commerce	RS	40596.22
agro_industry_and_commerce	SC	47.62
agro_industry_and_commerce	SP	33368.19
agro_industry_and_commerce	All States	118730.61
air_conditioning	DF	4297.84
air_conditioning	ES	3500.07
air_conditioning	MG	5359.74
air_conditioning	MS	184.99
air_conditioning	PR	9508.89
air_conditioning	RJ	13619.33
air_conditioning	RS	5333.59
air_conditioning	SC	1823.84
air_conditioning	SP	47542.37
air_conditioning	All States	91170.66

Year-over-Year Comparison using LAG()

LAG() compares each quarter's figures to the same quarter a year earlier, tracking growth trends across product categories and revealing seasonal patterns. Consistent quarter-over-quarter growth in a category signals market expansion and supports inventory planning, strategic decisions, and identifying which categories are growing or declining.



	CATEGORY	YEAR	QUARTER	REVENUE	PRIOR_YEAR_REVENUE	YOY_PCT_CHANGE
0	agro_industry_and_commerce	2017	Q1	610.97	NaN	NaN
1	agro_industry_and_commerce	2018	Q1	25390.24	610.97	4055.73
2	agro_industry_and_commerce	2017	Q2	3185.79	NaN	NaN
3	agro_industry_and_commerce	2018	Q2	12059.93	3185.79	278.55
4	agro_industry_and_commerce	2017	Q3	5064.88	NaN	NaN
...	...	...	...	...	...	...
464	watches_gifts	2017	Q2	103707.65	NaN	NaN
465	watches_gifts	2018	Q2	368971.06	103707.65	255.78
466	watches_gifts	2017	Q3	143000.27	NaN	NaN
467	watches_gifts	2018	Q3	195537.41	143000.27	36.74
468	watches_gifts	2017	Q4	274236.22	NaN	NaN

469 rows × 6 columns

Ranking within a Dimension

Window functions such as RANK() are evaluated after the WHERE clause runs, so a query cannot filter directly on a rank alias in the same SELECT. A CTE is used to calculate the rank first, then the outer query filters on that result.

	CATEGORY	SELLER_ID	SELLER_STATE	TOTAL_REVENUE	SELLER_RANK
0	agro_industry_and_commerce	e59aa562b9f8076dd550fcddf0e73491	PR	30323.68	1
1	agro_industry_and_commerce	6bd69102ab48df500790a8cecfc285c2	SP	2552.95	2
2	agro_industry_and_commerce	6481e96574816ead57975da2c0f6d80d	SP	2302.43	3
3	agro_industry_and_commerce	cfd7ddab722b902f7ac5b5f3ba6d723d	MG	2268.88	4
4	agro_industry_and_commerce	d17f467e4bf608a510c20d82f4ba3b6b	RS	2253.82	5
5	air_conditioning	fcdd820084f17e9982427971e4e9d47f	SP	17833.42	1
6	air_conditioning	6a51fc556dab5f766ced6fbc860bc613	SP	3677.62	2
7	air_conditioning	7a241947449cc45dbfda4f9d0798d9d0	MG	3542.41	3
8	air_conditioning	15aac934c58d886785ac1b17953ea898	ES	3500.07	4
9	air_conditioning	f3b80352b986ab4d1057a4b724be19d0	DF	2227.91	5
10	art	c31eff8334d6b3047ed34bebd4d62c36	SP	13405.89	1
11	art	955fee9216a65b617aa5c0531780ce60	SP	1067.70	2
12	art	b561927807645834b59ef0d16ba55a24	SP	836.90	3

OLAP Operation Identification

Operation 1

Operation: **SLICE (Time Dimension)**

A slice filters on a single dimension. This query filters only on time (WHERE t.year = 2018 AND t.quarter = 'Q1'), taking one flat cross-section of the cube — Q1 2018 — while other dimensions remain unrestricted. The single-dimension WHERE clause is what identifies this as a slice.

	year	quarter	category_english	revenue
0	2018	Q1	watches_gifts	289840.57
1	2018	Q1	furniture_decor	292027.86
2	2018	Q1	telephony	84126.47
3	2018	Q1	fashion_shoes	4303.71
4	2018	Q1	bed_bath_table	351036.09
..	...	...	...	...
64	2018	Q1	home_comfort_2	184.33
65	2018	Q1	la_cuisine	556.73
66	2018	Q1	diapers_and_hygiene	2568.10
67	2018	Q1	fashio_female_clothing	859.65
68	2018	Q1	fashion_childrens_clothes	216.87

Operation 2

Operation: **SLICE with drill-down (Customer State -> City)**

This query filters on a single dimension (WHERE c.customer_state = 'SP'), taking a slice showing only customers from São Paulo. Within that slice it then drills down from state to city level, adding detail within the same geographic hierarchy without introducing a new dimension.

Query — Slice (customer_state = 'SP'), drilled down to city

	customer_state	customer_city	total_orders	total_revenue
0	SP	sao paulo	15402	2170227.12
1	SP	campinas	1429	212541.70
2	SP	guarulhos	1178	163575.82
3	SP	sao bernardo do campo	928	119024.85
4	SP	santos	706	111670.21
...	...	...	...	...
623	SP	boraceia	1	39.96
624	SP	nova independencia	1	34.86
625	SP	nova luzitania	1	34.75
626	SP	aparecida d'oeste	1	31.75
627	SP	ibitiuva	1	27.69

Operation 3

Operation: **DICE (State, Category, Year)**

A dice filters on multiple dimensions at once. This query filters simultaneously on customer_state, product category, and year (customer_state IN (...), category IN (...), and year =

2018), carving out a smaller "sub-cube" of the data. This is more restrictive than a slice and supports focused analysis on a specific combination of values across dimensions.

```
Query – Dice (3 dimensions: state, category, year)
```

	customer_state	product_category	year	total_orders	total_revenue
0	MG	computers_accessories	2018	549	78752.84
1	MG	electronics	2018	163	18214.42
2	RJ	computers_accessories	2018	504	72886.69
3	RJ	electronics	2018	276	21568.68
4	SP	computers_accessories	2018	1701	241075.14
5	SP	electronics	2018	646	40631.20

Operation 4

Operation: **PIVOT**

A pivot reshapes rows into columns, typically using CASE WHEN ... THEN ... END:

SUM(CASE WHEN c.customer_state='SP' THEN f.payment_value ELSE 0 END) AS SP, SUM(CASE WHEN c.customer_state='RJ' THEN f.payment_value ELSE 0 END) AS RJ, SUM(CASE WHEN c.customer_state='MG' THEN f.payment_value ELSE 0 END) AS MG

This transforms customer_state row values (SP, RJ, MG) into separate columns, creating a cross-tabulation where each category has one row with revenue spread across state columns rather than multiple rows per category. The CASE statements are what identify this as a pivot.

Query 4 – Pivot (states as columns)

	product_category	revenue_sp	revenue_rj	revenue_mg
0	agro_industry_and_commerce	27789.48	9385.32	12661.48
1	air_conditioning	25447.92	18082.58	4827.45
2	art	16480.32	3654.50	1699.96
3	arts_and_craftmanship	1090.77	0.00	627.35
4	audio	19116.08	10331.91	6234.00
...	...	...	...	...
66	tablets_printing_image	2752.47	1059.27	996.24
67	telephony	121368.16	45619.00	42960.55
68	toys	210953.86	84533.13	68634.00
69	unknown	77565.41	28223.74	22770.79
70	watches_gifts	462729.65	199245.11	134650.91

PIVOT = Rows >>> Columns

Before PIVOT

Category	State	Revenue
Electronic	SP	$50000
Electronic	RJ	$30000
Electronic	MG	$20000
Furniture	SP	$40000
Furniture	RJ	$25000
Furniture	MG	$15000

After PIVOT View:

Category	SP	RJ	MG
Electronics	$50000	$30000	$20000
Furnitures	$40000	$25000	$15000

ETL Pipeline Design

Load order

Order of the table

1. DIM_TIME
2. DIM_CUSTOMER
3. DIM_PRODUCT
4. DIM_SELLER
5. FACT_ORDERS

This order is a must because FACT_ORDERS has foreign key constraints that reference all dimension tables, so dimensions must exist and have data before the fact table can be loaded to maintain referential integrity.

CDC mechanism

Mechanism: Timestamp-based incremental load using order_purchase_timestamp

CDC (Change Data Capture) identifies and extracts only new or changed data. Since new orders arrive daily and existing orders are never deleted, order_purchase_timestamp provides a reliable way to extract only records newer than the last load, significantly reducing extraction time and resource usage compared to a full reload each cycle.

Limitation: this mechanism does not capture updates to existing orders (e.g., a status change from "processing" to "delivered"), which would require an additional CDC method such as triggers or log-based capture.

Example:

- Day 1 (initial load): all orders up to 2018-01-01 23:59:59 are loaded; last_loaded_timestamp is set to 2018-01-01 23:59:59.

- Day 2 (incremental load): SELECT * FROM orders WHERE order_purchase_timestamp > '2018-01-01 23:59:59' returns only Day 2's orders; last_loaded_timestamp is updated to 2018-01-02 23:59:59.

Data Quality Problems

Problem 1 — Missing product category translations: some categories in olist_products_dataset.csv have no match in product_category_name_translation.csv, leaving category_eng NULL for those rows.

Fix: LEFT JOIN combined with COALESCE, falling back to the Portuguese name. Left unresolved, these NULLs would cause incomplete category analysis and broken GROUP BY results.

product_category_name	English Translation	category_eng (After Fix)
pc_gamer	NULL	pc_gamer

Problem 2 — Null values in product numeric columns: product_weight_g, product_length_cm, product_height_cm, and product_width_cm have missing values for some products.

Fix: replace NULLs with 0 using fillna(0) in Python or COALESCE in SQL.

DIM_TIME loading

DIM_TIME is loaded once — typically as an initial bulk load, refreshed annually (e.g., on January 1st) to extend the date range. A date-spine generator script produces every date across a wide range (e.g., 2000–2050) along with derived attributes such as day of week, month, quarter, and holiday flag.

DIM_TIME differs from other dimensions in two ways:

- One table, many uses (role-playing): a single order has multiple dates — ordered, shipped, delivered. DIM_TIME is loaded once and reused via separate foreign keys in the fact table instead of creating three separate time tables.
- The table never changes: unlike customers (who move) or products (which get updated), dates are fixed — January 1st, 2025 is always January 1st, 2025 — so it only needs to be loaded in bulk, not maintained.

Example — the same DIM_TIME table referenced via three different foreign keys:

ORDER_DATE_SK	FULL_DATE	DAY_NAME	MONTH	YEAR
101	Jan 1	Monday	Jan	2025
102	Jan 2	Tuesday	Jan	2025
103	Jan 3	Wednesday	Jan	2025

FACT_ORDERS (using the same table with different IDs):

ORDER_ID	ORDER_DATE_SK	SHIP_DATE_SK	DELIVERY_DATE_SK
5001	101	103	103

Physical Design Decisions

Partitioning strategy

Partitioning

Range partitioning on ORDER_DATE_SK, splitting FACT_ORDERS by year

```
# --- C8.1 — Partitioning Strategy ---
# Range partitioning on ORDER_DATE_SK, splitting FACT_ORDERS by year
# order_id is VARCHAR2(50) to match the real FACT_ORDERS table (Olist order IDs are text, not numbers)

ddl_partition = """
CREATE TABLE FACT_ORDERS_PART (
    order_id          VARCHAR2(50),
    order_date_sk     NUMBER,
    customer_key      NUMBER,
    product_key       NUMBER,
    seller_key        NUMBER,
    price             NUMBER,
    freight_value     NUMBER
)
PARTITION BY RANGE (order_date_sk) (
    PARTITION p_2016 VALUES LESS THAN (20170101),
    PARTITION p_2017 VALUES LESS THAN (20180101),
    PARTITION p_2018 VALUES LESS THAN (20190101),
    PARTITION p_max  VALUES LESS THAN (MAXVALUE)
)
"""

with engine.connect() as conn:
    conn.execute(text(ddl_partition))
    conn.commit()
print("Partitioned table created.")
```

```
[66]  ✓ 0.0s

...  Partitioned table created.
```

Use range partitioning on the ORDER_DATE_SK (or ORDER_DATE if it is a date column) column in the FACT_ORDER table. This drastically reduces I/O and speeds up the query.

```sql
SELECT
    TO_CHAR(order_date, 'YYYY-MM') AS month,
    SUM(order_amount) AS total_sales
FROM
    fact_orders
WHERE
    order_date BETWEEN DATE '2025-01-01' AND DATE '2025-03-31'
GROUP BY
    TO_CHAR(order_date, 'YYYY-MM');
```

Bitmap index selection

1. CUSTOMER_COUNTRY_SK (or COUNTRY_ID) in FACT_ORDERS — approximate cardinality: low (200 distinct countries).

Bitmap is suitable here: a B-Tree index on a low-cardinality column is inefficient, since it returns too many rows per key and makes index scans expensive. A bitmap index compresses the data and enables fast bitwise AND/OR operations when combined with other filters.

2. ORDER_STATUS_SK (or STATUS_CODE) in FACT_ORDERS — approximate cardinality: very low (5–10 distinct statuses, e.g., Pending, Shipped, Delivered, Cancelled).

Bitmap is again the better fit. Fact tables are read-intensive with very few UPDATE/DELETE operations (which can lock bitmap segments), so in a data warehouse setting the low DML overhead makes bitmap indexes safe and highly effective for ad-hoc queries filtered by order status.

Materialized View

SQL Plus before create materialized view,

ALTER USER system IDENTIFIED BY dwh12345;

GRANT CREATE MATERIALIZED VIEW TO olist_dwh;

exit;

```
SQL> ALTER SESSION SET CONTAINER = XEPDB1;

Session altered.

SQL> ALTER USER system IDENTIFIED BY dwh12345;
ALTER USER system IDENTIFIED BY dwh12345
*
ERROR at line 1:
ORA-65066: The specified changes must apply to all containers


SQL> GRANT CREATE MATERIALIZED VIEW TO olist_dwh;

Grant succeeded.

SQL> exit;
```

```
    with engine.connect() as conn:
        conn.execute(text(ddl_mv))
        conn.commit()
    print("Materialized view created.")
3]  ✓ 2.5s

Materialized view created.
```

```
df_mv_data = pd.read_sql("""
    SELECT * FROM mv_daily_sales_by_category
    ORDER BY full_date, category_english
    FETCH FIRST 10 ROWS ONLY
""", engine)
display(df_mv_data)
```

✓ 0.1s

	full_date	category_english	order_count	total_sales_amount	total_payment_amount
0	2016-09-04	furniture_decor	1	136.23	272.46
1	2016-09-05	telephony	1	75.06	75.06
2	2016-09-15	health_beauty	1	143.46	NaN
3	2016-10-02	baby	1	109.34	109.34
4	2016-10-03	fashion_shoes	1	40.95	40.95
5	2016-10-03	furniture_decor	2	225.73	225.73
6	2016-10-03	sports_leisure	3	128.43	128.43

REFLECTION

1. A difficult design decision

The hardest decision was choosing an SCD strategy for state in DIM_SELLER. A seller's registered state (seller_state in olist_sellers_dataset.csv) can change if they change the location, and I had three options.

Type 1 (overwrite) was simplest but would retroactively rewrite history a seller's 2017 revenue would appear under their new 2019 state, corrupting any historical "revenue by state" report.

Type 3 (add a previous_state column) was tempting for its simplicity but only stores one prior value, failing if a seller moves more than once.

Finally I chose Type 2, adding effective_date, expiry_date, and is_current columns to DIM_SELLER, because it preserves a complete, queryable history of every state change while is_current = 'Y' still gives a fast "current state" lookup. The tradeoff was more ETL logic to detect and version changes, but it's the only option that keeps historical OLAP queries (like B1's state-level ROLLUP) accurate over time.

2. Schema Comparison

My Star Schema:

- DIM_CUSTOMER (combines customers + geolocation)
- DIM_PRODUCT (combines products + category translation)
- DIM_SELLER (with geolocation)
- DIM_TIME (role-playing)
- FACT_ORDERS (order_items + orders + payments + reviews)

Everything is consolidated into one fact table at order-item grain plus four dimension tables, using integer surrogate keys instead of text IDs for faster joins. Portuguese category names are translated to English during the ETL load, so users don't need to repeat that translation on every query.

Speed and simplicity: Query B1 (sales by category and state with subtotals) now needs just one ROLLUP over a two-table join. Originally, it required joining five different files with complex logic every single time. Integer joins are also much faster than joining on long text strings across 100,000 plus order items.

Flexibility and completeness: My schema is fixed. it only covers orders, products, customers, sellers, and time. If someone wants to analyze review comments or use geographic coordinates, they can't because I didn't build dimensions for reviews or geolocation data. They'd have to go back to the original CSV files.

3. A Real Data Quality Issue

While profiling olist_products_dataset.csv against product_category_name_translation.csv, I found two Portuguese category names with no English match: portateis_cozinha_e_preparadores_de_alimentos and pc_gamer. In C7.3, I handled this during the ETL transform phase by falling back to the original Portuguese name (rather than leaving category_eng NULL) whenever the translation join found no match, so these products still populate DIM_PRODUCT with a usable category label instead of a blank field.

Had I ignored this, any query grouping or filtering on category_eng — including my B1 ROLLUP and B3 seller-ranking query — would either silently drop these products from an inner join or bucket them under a NULL group, understating total revenue for those two product lines. It's a small issue (two categories out of ~70) but exactly the kind of silent, easy-to-miss bug that produces confidently wrong dashboard numbers.

Dataset: [Brazilian E-Commerce Public Dataset by Olist]

Power BI Implementation